SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta elektrotechniky a informatiky

Klient-Server aplikácia na spracovanie obrazu očných sietnic

Tímový projekt

Riešitelia: Bc. Christán Žilinčár, Bc. Michal Metke, Bc. Samuel Mrekaj, Bc. Jakub Tinák, Bc. Lukáš Matejov, Bc. Martin Mizner

2025

STU FEI
SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE
FAKULTA ELEKTROTECHNIKY A INFORMATIKY
ÚSTAV ROBOTIKY A KYBERNETIKY

TÍMOVÝ PROJEKT

ZADANIE

Študijný program:	**Robotika a kybernetika**
Študijný odbor:	**kybernetika**
Vedúci projektu:	**Kajan, Körösi**
Miesto vypracovania projektu:	**Ústav robotiky a kybernetiky**
Riešitelia:	

Názov projektu: **Klient-Server aplikácia na spracovanie obrazov očných sietnic**

Špecifikácia zadania:
Cieľom aplikácie je na strane servera ukladať, spravovať a spracovávať obrazy očných sietnic a na strane klienta cez web rozhranie pristupovať k databáze obrazov sietnic, spúšťať algoritmy na spracovanie obrazov a prezerať si výsledky spracovania obrazov.

Úlohy:

1. Vytvorte Server aplikáciu s MySQL databázou (*obsahujúca úrovne používateľov, anonymizované údaje o pacientoch, cesty k súborom obrazov, ...)*
2. Vytvorte web rozhranie pre správu databázy na strane servera. *(zobrazenie obrazov a výsledkov spracovania podľa užívateľov, ...)*
3. Vytvorte web formulár pre pristup na server z mobilného zariadenia *(pristup PWA – progressive web apps)*.
4. Zabudujte do aplikácie možnosť spúšťať preddefinované algoritmy klasifikácie, segmentácie a detekcie na vybraný obraz očnej sietnice.
5. Vykonajte testovanie jednotlivých funkcionalít aplikácie.

Termín odovzdania projektu: ***20.6.2025***

V Bratislave dňa 17.2.2025

prof. Ing. Jarmila Pavlovičová, PhD.
garantka študijného programu

Obsah

Obsah 2
1 Úvod 3
2 Architektúra systému 4
2.1 Frontend technológie 4
2.2 Backend technológie 6
2.2.1 Flask Framework 6
2.3 Databázový model 7
2.4 API Endpointy 10
2.4.1 Implementované endpointy 11
2.4.2 Správa používateľov 13
3 Spracovanie obrazu 15
4 Architektúra frontendu 17
5 Screenshoty z aplikácie 19
6 Bezpečnosť 24
7 Inštalácia a konfigurácia 25
8 Záver 26

1 Úvod

V dnešnej dobe, keď sa medicínska diagnostika čoraz viac opiera o digitálne zobrazovacie metódy, prinášame webovú aplikáciu na spracovanie obrazov očných sietnic, ktorá spája jednoduché ovládanie so spoľahlivou ochranou citlivých dát. Po prihlásení sa používateľovi zobrazí prehľadné rozhranie, v ktorom môže inteligentne nahrávať snímky vo formáte JPG alebo PNG – systém ihneď overí ich platnosť, skontroluje vyplnenie povinných údajov (pacient, oko, kvalita snímky) a zobrazí náhľad s možnosťou zrušenia nahrávania bez obnovenia stránky. Ak je to žiaduce, používateľ súčasne zvolí jednu alebo viac metód spracovania (napríklad detekciu či segmentáciu pomocou AI), inak sa snímka iba archivuje na serveri.

Na serverovej strane sa každý súbor bude ukladať do štruktúrovaného adresára podľa pacienta a typu snímky, pričom sa v databáze vytvorí záznam s kompletnými metadátami: od informácií o zariadení a autorovi až po kvalitu fotografie. Po odoslaní spracovacích požiadaviek aplikácia sleduje ich stav, ukladajú sa časové značky a výsledky sú automaticky priraďované k pôvodným snímkam. Používateľ potom môže vo webovej galérii filtrovať a triediť obrázky, porovnávať originál s výstupmi jednotlivých algoritmov a prezerať si detailné diagnostické údaje vrátane surových JSON výstupov.

Celý systém je postavený na modernom, trojvrstvovom princípe: na frontende s responzívnym návrhom na báze Jinja2 šablón, na Flask backende s REST API, autentifikáciou cez JWT a modulmi pre logovanie i monitoring, a na spoľahlivej PostgreSQL databáze spravovanej cez SQLAlchemy. Vďaka optimalizáciám, ako sú lazy loading obrázkov, minifikácia statických súborov a browser caching, aplikácia reaguje rýchlo aj pri vysokom zaťažení. Role-based prístupový systém a šifrované spojenie cez HTTPS zabezpečujú, že citlivé dáta pacientov zostávajú chránené, a zároveň umožňujú jednoduché rozšírenie či prepojenie s inými systémami podľa potrieb klinickej praxe.

2 Architektúra systému

Systém je postavený na trojvrstvovej architektúre:

1. **Frontend**
- Webové rozhranie s responzívnym dizajnom
- HTML šablóny s použitím Jinja2
- Statické súbory (CSS, JavaScript, obrázky)
- Bootstrap komponenty pre UI

2. **Backend**
- Flask server
- REST API
- Autentifikačný systém
- Spracovateľské moduly
- Logovanie a monitoring

3. **Databáza**
- PostgreSQL databáza
- SQLAlchemy ORM
- Migračný skript
- Zálohovanie dát

2.1 Frontend technológie

Frontendová časť systému je založená na štandardných webových technológiách – HTML5, CSS3 a JavaScript. Pre dynamický obsah je použitý šablónovací systém Jinja2, ktorý umožňuje efektívne generovanie HTML stránok na základe dát z backendu. Bootstrap framework poskytuje systém pre responzívny dizajn a predpripravené komponenty, čo uľahčuje tvorbu používateľsky dostupného rozhrania.

Frontend architektúra je rozdelená na logické komponenty, ktoré oddeľujú zodpovednosti – navigácia, zobrazenie galérie obrazov, interakcia s formulármi a vizualizácia výsledkov spracovania. Každý komponent je navrhnutý s dôrazom na prehľadnosť kódu. Používateľské rozhranie bolo navrhnuté v súlade s princípom mobile-first, a prispôsobuje sa rôznym veľkostiam zariadení pomocou techník Flexbox a Grid Layout.

Štruktúra frontend časti je organizovaná do nasledujúcich adresárov:

- `templates/` - obsahuje HTML šablóny pre jednotlivé stránky
- `static/css/` - štýlové súbory a definície vzhľadu
- `static/js/` - JavaScript súbory pre interaktivitu
- `static/icons/` - ikony a grafické prvky

Používateľské rozhranie a komponenty

Hlavné komponenty používateľského rozhrania zahŕňajú navigačný systém, formuláre a zobrazenie dát. Navigačný systém poskytuje responzívny panel s podporou mobilných zariadení a dynamické menu s prístupovými právami. Formuláre obsahujú validáciu v reálnom čase a sú implementované pre prihlasovanie, registráciu a správu pacientov.

Zobrazenie dát je realizované prostredníctvom interaktívnej galérie obrazov s možnosťami filtrovania a triedenia. Dashboard poskytuje prehľad aktuálnych úloh a základných štatistík. Detailné zobrazenie výsledkov spracovania obrazov umožňuje efektívnu analýzu a interpretáciu dát.

Responzívny dizajn a prístupnosť

Pre zabezpečenie konzistencie vzhľadu naprieč aplikáciou sa využívajú globálne definované štýly v CSS. Doplnkové štýly sú prispôsobené tak, aby vyhovovali požiadavkám prístupnosti a kompatibility s rôznymi zariadeniami. Formuláre obsahujú mechanizmy validácie vstupov, čím sa minimalizujú chyby a zvyšuje používateľský komfort.

Responzívny dizajn je implementovaný pomocou media queries pre rôzne veľkosti obrazoviek, Flexbox layoutu pre pružné rozloženie prvkov a Grid systému pre

komplexnejšie rozloženia. Používajú sa relatívne jednotky (rem, em) pre škálovateľnosť a konzistentný vzhľad naprieč rôznymi zariadeniami.

Interaktivita a dynamické správanie

JavaScript kód je organizovaný do modulov podľa funkcionality. Implementované sú funkcie pre správu formulárov a validáciu, asynchrónne načítavanie dát a interaktívne galérie. Pre komunikáciu s backendom sa využívajú asynchrónne HTTP požiadavky, ktoré umožňujú dynamické aktualizácie obsahu bez obnovenia stránky.

Výkon

Aktuálne sú implementované základné optimalizácie výkonu, ktoré zahŕňajú lazy loading obrázkov v galérii, minifikáciu statických súborov a browser caching pre statické assets. Tieto optimalizácie zlepšujú rýchlosť načítavania a celkový výkon aplikácie.

2.2 Backend technológie

Serverová časť systému je implementovaná vo frameworku Flask v jazyku Python. Flask poskytuje nástroje na definovanie REST API rozhraní a prácu s požiadavkami klienta. Je vhodný pre menšie a stredne veľké aplikácie, kde je dôležitá rýchla implementácia a možnosť flexibilného rozšírenia.

2.2.1 Flask Framework

Flask je ľahký a flexibilný webový framework pre Python, ktorý poskytuje základné nástroje pre vývoj webových aplikácií. Je založený na WSGI (Web Server Gateway Interface) štandarde a Werkzeug toolkite, čo mu umožňuje efektívne spracovávať HTTP požiadavky a odpovede. Framework je navrhnutý s dôrazom na jednoduchosť a flexibilitu, čo umožňuje vývojárom implementovať vlastné riešenia bez zbytočných obmedzení.

Implementácia Flasku v projekte

V našom projekte je Flask implementovaný s dôrazom na modulárnosť a škálovateľnosť. Aplikačná štruktúra je rozdelená do logických častí, kde každý modul má jasne definovanú zodpovednosť. Hlavné komponenty zahŕňajú inicializáciu aplikácie, konfiguráciu, databázové modely, API endpointy a business logiku.

Konfigurácia aplikácie je rozdelená do niekoľkých úrovní, od základných nastavení až po runtime konfiguráciu pre špecifické prostredia. Bezpečnostné nastavenia zahŕňajú šifrovanie citlivých dát, nastavenie CORS politík a konfiguráciu JWT tokenov.

Routing systém je implementovaný pomocou Flask Blueprintov, čo umožňuje logické zoskupenie súvisiacich endpointov a lepšiu organizáciu kódu. Každý endpoint je implementovaný s dôrazom na validáciu vstupných parametrov, správne HTTP status kódy a konzistentný formát odpovedí.

Flask Extensions

Projekt využíva niekoľko kľúčových Flask rozšírení. Flask-SQLAlchemy poskytuje elegantné rozhranie pre prácu s databázou, umožňujúc definovať databázové modely pomocou Python tried. Flask-JWT-Extended implementuje JWT autentifikáciu pre bezpečnú a škálovateľnú správu používateľských relácií.

Správa databázových migrácií je založená na Flask-Migrate, ktoré umožňuje sledovať zmeny v databázovom schéme a automaticky generovať migračné skripty. Pre ochranu API pred nadmerným zaťažením používame Flask-Limiter, ktorý implementuje rate limiting pre zabránenie DoS útokom a zneužitiu API.

2.3 Databázový model

Systém využíva PostgreSQL databázu, ktorá poskytuje robustné riešenie pre ukladanie a správu dát. Databáza je navrhnutá s dôrazom na integritu dát, výkon a bezpečnosť. Pre prácu

s databázou je použitý SQLAlchemy ORM, ktorý poskytuje elegantné rozhranie pre definovanie modelov a vykonávanie databázových operácií.

Hlavné entity

Databázový model je založený na niekoľkých kľúčových entitách, ktoré spolu tvoria komplexný systém pre správu retinálnych obrazov a súvisiacich dát. Každá entita je navrhnutá s ohľadom na normalizáciu dát a efektívne vzťahy medzi tabuľkami.

- **Hospitals**

Entita Hospitals je navrhnutá pre správu nemocníc v systéme. Obsahuje základné informácie o nemocnici ako názov, adresa (krajina, mesto, ulica, PSČ) a unikátne kód pre adminov, doktorov a technikov. Každá nemocnica môže mať viacerých členov. Entita je prepojená s entitami Admin_data, Doctor_Data a Technician_Data pre správu personálu.

- **Users**

Správa používateľských účtov je implementovaná pomocou entity Users, ktorá uchováva všetky potrebné informácie o používateľoch systému. Heslá sú bezpečne hashované pomocou bcrypt algoritmu. Každý používateľ má priradenú rolu, ktorá určuje jeho oprávnenia v systéme. Entita obsahuje základné informácie ako email, meno, priezvisko a čas posledného prihlásenia.

- **Super Admins**

Super Admins je navrhnutá pre správu systému. Super administrátori majú najvyššie oprávnenia na serveri a môžu vytvárať nemocnice, používateľov a metódy.

- **Admins**

Entita Admins je navrhnutá pre správu systému pre každú nemocnicu. Administrátori majú najvyššie oprávnenia v nemocnici. Môže vytvárať len doktorov a technikov v jeho nemocnici.

- **Doctors**

Entita Doctors je navrhnutá pre správu lekárov v systéme. Každý lekár má priradené základné osobné údaje ako meno, priezvisko, telefónne číslo a pohlavie. Lekár môže mať priradený titul a príponu. Každý lekár patrí konkrétnej nemocnici a môže mať priradených viacerých pacientov. Entita je prepojená s entitou Hospital a s entitou Patients pre správu pacientov.

- **Technicians**

Entita Technicians je navrhnutá pre správu technikov v systéme. Každý technik má priradené základné osobné údaje ako meno a priezvisko. Technik patrí konkrétnej nemocnici. Entita je prepojená s entitou Hospital pre správu nemocníc.

- **Patients**

Entita Patients je navrhnutá pre ukladanie anonymizovaných údajov o pacientoch v súlade s GDPR a ďalšími zdravotníckymi reguláciami. Obsahuje demografické údaje ako vek, pohlavie a základné zdravotné informácie.

- **Devices**

Entita Devices je navrhnutá pre správu zariadení používaných na zachytávanie retinálnych obrazov. Obsahuje informácie o zariadení ako názov, typ, model kamery a rozlíšenie. Každé zariadenie môže byť spojené s viacerými originálnymi obrazmi. Entita je prepojená s entitou OriginalImageData.

- **OriginalImages**

Entita OriginalImages je navrhnutá pre správu originálnych retinálnych obrazov. Obsahuje metadáta o obraze ako cesta k súboru, kvalita, dátum vytvorenia, ktoré oko, diagnóza a cestu k spracovaným fotkám. Každý obraz je spojený s konkrétnym zariadením, ktoré ho vytvorilo, používateľom, ktorý ho nahral, a pacientom, ktorému patrí. Entita je prepojená s entitami DeviceData, User, PatientData a ProcessedImageData.

- **ProcessedImages**

Entita ProcessedImages je navrhnutá pre správu spracovaných retinálnych obrazov. Obsahuje informácie o spracovaní ako dátum vytvorenia, stav spracovania, technické a diagnostické poznámky, typ spracovania a cesty k spracovaným súborom. Každý spracovaný obraz je spojený s originálnym obrazom. Entita je prepojená s entitou OriginalImageData.

Vzťahy a integrita

Medzi entitami sú definované vzťahy, ktoré zabezpečujú referenčnú integritu dát. Používajú sa cudzie kľúče s kaskádovým mazaním tam, kde je to vhodné, a s obmedzením tam, kde je potrebná dodatočná kontrola. Všetky vzťahy sú indexované pre optimalizáciu výkonu pri zložitých dotazoch.

Indexovanie a optimalizácia

Databáza je optimalizovaná pre rýchle vyhľadávanie a efektívne spracovanie dát. Používajú sa B-tree indexy pre textové a číselné polia, ktoré sú často používané v dotazoch. Často používané dotazy sú optimalizované pomocou čiastočných indexov.

2.4 API Endpointy

Systém je postavený na REST (Representational State Transfer) API architektúre, ktorá poskytuje štandardizovaný spôsob komunikácie medzi klientom a serverom. REST API je bezstavové, čo znamená, že každá požiadavka obsahuje všetky potrebné informácie pre jej spracovanie.

Princípy REST API

API je navrhnuté podľa kľúčových princípov REST architektúry. Každý endpoint reprezentuje konkrétny zdroj (používateľ, pacient, obraz) a používa štandardné HTTP

metódy (GET, POST, PUT, DELETE) pre operácie s dátami. Odpovede obsahujú vhodné HTTP stavové kódy pre indikáciu výsledku operácie a dáta sú vymieňané v štandardizovanom JSON formáte.

Bezpečnosť a autentifikácia

Všetky API endpointy sú chránené pomocou JWT (JSON Web Token) autentifikácie. Token je potrebné poskytnúť v hlavičke Authorization každej požiadavky. Systém implementuje základný rate limiting pre ochranu pred nadmerným zaťažením. Citlivé dáta sú prenášané len cez HTTPS protokol.

2.4.1 Implementované endpointy

Systém poskytuje nasledujúce hlavné endpointy:

Autentifikácia a registrácia

- GET /login - Zobrazenie prihlasovacieho formulára
- POST /login - Prihlásenie používateľa (rate limit: 5 pokusov za 5 minút)
- GET /register - Zobrazenie výberu typu registrácie
- GET /register/{user_type} - Zobrazenie registračného formulára pre daný typ používateľa
- POST /register/{user_type} - Registrácia nového používateľa (rate limit: 3 pokusy za minútu)

Správa účtu

- GET /account - Zobrazenie stránky účtu
- GET /account/info - Získanie informácií o účte
- POST /account/edit - Úprava údajov účtu

Správa nemocníc (len pre admina)

- GET /api/hospital/list - Získanie zoznamu nemocníc

- POST /api/hospital/add - Pridanie novej nemocnice
- PUT /api/hospital/update/{hospital_id} - Aktualizácia nemocnice

Správa pacientov (pre lekárov)
- POST /doctor/assign - Priradenie pacienta k lekárovi
- POST /doctor/remove - Odstránenie pacienta od lekára
- POST /doctor/transfer - Presun pacienta k inému lekárovi
- POST /doctor/change_hospital - Zmena nemocnice

Formát odpovedí

Systém podporuje dvojitý formát odpovedí na základe hlavičky Accept:
- application/json - JSON odpoveď pre API klientov
- text/html - HTML odpoveď pre webové rozhranie

Každá odpoveď obsahuje:
- HTTP status kód indikujúci výsledok operácie
- Telo odpovede v požadovanom formáte (JSON/HTML)
- V prípade chyby obsahuje chybovú správu

Validácia a error handling

Všetky endpointy implementujú základnú validáciu vstupov:
- Kontrola formátu vstupných dát (JSON/formulár)
- Validácia povinných polí
- Kontrola oprávnení používateľa
- Rate limiting pre citlivé endpointy

2.4.2 Správa používateľov

Systém implementuje hierarchický systém rolí s jasne definovanými oprávneniami pre každú rolu:

- **Superadmin**

  Superadmin má absolútne oprávnenia v systéme. Môže:
  - Správu všetkých nemocníc v systéme
  - Vytváranie a správu admin účtov
  - Konfiguráciu systému
  - Pridávanie a správu nových nemocníc
  - Prístup k všetkým dátam v systéme
  - Správa všetkých používateľov
  -Správa metód

- **Administrátor**

  Administrátor má najvyššie oprávnenia v rámci svojej nemocnice. Môže:
  - Správa používateľských účtov v rámci svojej nemocnice
  - Konfigurácia nastavení systému pre svoju nemocnicu
  - Prístup k auditným záznamom

- **Super doktor**

  Super doktor má rozšírené oprávnenia oproti bežnému lekárovi. Môže:
  - Správa všetkých pacientov
  - Prístup k všetkým retinálnym obrazom
  - Generovanie reportov pre všetkých pacientov
  - Presúvanie pacientov medzi lekármi

  - **Doktor**

  Lekár je primárny používateľ systému s rozsiahlymi oprávneniami. Môže:
  - Správa svojich pacientov
  - Nahrávanie a správa retinálnych obrazov
  - Generovanie reportov

- Komunikácia s pacientmi
- Pridávanie poznámok k vyšetreniam

– **Technik**

Technik má špecializované oprávnenia pre prácu s obrazovými dátami. Môže:
- Nahrávanie retinálnych obrazov
- Komunikácia s lekármi a adminom nemocnice
-Vytváranie pacientov

– **Pacient**

Pacient má obmedzené oprávnenia na prístup k vlastným dátam. Môže:
- Komunikácia s lekárom
- Správa vlastného profilu

Správa oprávnení

Systém používa granularný systém oprávnení, ktorý umožňuje:
- Presnú kontrolu prístupu pre každú rolu
- Dynamické pridávanie a odoberanie oprávnení
- Logovanie všetkých zmien oprávnení
- Automatickú deaktiváciu neaktívnych účtov
- Kontrolu prístupu k citlivým dátam

3 Spracovanie obrazu

Spracovanie medicínskych snímok je hlavnou súčasťou tejto aplikácie, ktorá umožňuje používateľom efektívne nahrávať, analyzovať a spravovať fotky pacientov. Celý proces je navrhnutý tak, aby bol intuitívny, bezpečný a poskytoval jasnú spätnú väzbu v každom kroku. V nasledujúcich sekciách je popísané, ako prebieha spracovanie fotiek od ich nahratia až po zobrazenie výsledkov.

- **Nahrávanie fotky**

Proces spracovania fotiek v aplikácii začína nahraním obrázka používateľom prostredníctvom webového rozhrania. Počas výberu súboru na strane klienta prebieha kontrola, či ide o podporovaný formát (JPG alebo PNG). Akonáhle používateľ vyberie obrázok, zobrazí sa mu jeho náhľad a má možnosť výber kedykoľvek zrušiť. Pred samotným odoslaním formulára sa automaticky kontroluje, či sú vyplnené všetky povinné polia, ako napríklad pacient, oko alebo kvalita snímky. Po úspešnej validácii sa fotka odošle na server pomocou AJAX požiadavky, čo zabezpečuje rýchlu a plynulú používateľskú skúsenosť bez zbytočného obnovovania stránky. Súčasťou tohto procesu , aké metódy spracovania sa majú na nahranú fotku aplikovať. Používateľ má na výber zo zoznamu dostupných metód (napríklad detekcia, segmentácia a podobne), ktoré môže podľa potreby zaškrtnúť. Ak si však žiadnu metódu nevyberie, fotka sa jednoducho uloží do systému bez ďalšieho spracovania. V takom prípade je fotka dostupná v zozname nahraných obrázkov, ale neprebehne žiadna analýza ani spracovanie na pozadí. Táto flexibilita umožňuje používateľovi rozhodnúť sa, či chce fotku len archivovať, alebo ju rovno odoslať na ďalšie spracovanie podľa aktuálnych potrieb.

- **Ukladanie fotky na serveri**

Na serverovej strane je prijatá fotka uložená do štruktúrovaného adresára, ktorý je organizovaný podľa pacienta a typu obrázka (originál alebo spracované). Zároveň sa vytvorí záznam v databáze, ktorý obsahuje všetky dôležité informácie o fotke, vrátane údajov o pacientovi, autorovi, použitom zariadení a kvalite snímky. Pri nahrávaní sa zaznamenáva aj informácia o zariadení, ktorým bola fotka vytvorená, vrátane typu a modelu. Týmto spôsobom je zabezpečená prehľadnosť a jednoduchá správa všetkých nahraných obrázkov.

- **Odoslanie na spracovanie pomocou AI**

Po úspešnom nahraní fotky môže používateľ zvoliť jednu alebo viac metód spracovania, ako napríklad detekciu alebo segmentáciu. V rozhraní je zobrazený zoznam dostupných metód, z ktorých si môže používateľ vybrať podľa potreby. Pred samotným odoslaním na spracovanie aplikácia automaticky overí, či je spracovací server dostupný. Pre každú zvolenú metódu sa vytvorí požiadavka, ktorá obsahuje identifikátor fotky, názov metódy a jej parametre, metadáta o pacientovi, oku, diagnóze, zariadení a samotný obrázok zakódovaný v base64 formáte. Pre každú požiadavku sa v databáze vytvorí záznam o spracovávanom obrázku so stavom „čaká na spracovanie“, čím je zabezpečený prehľad o aktuálnom stave spracovania.

- **Spracovanie a prijatie výsledkov**

Spracovací server prijme požiadavku, vykoná požadovanú metódu a výsledok – či už ide o nový obrázok, masku alebo iné výstupné dáta – odošle späť na server aplikácie. Backendová časť aplikácie prijme výsledok, uloží spracovaný obrázok do správneho adresára a aktualizuje záznam v databáze, pričom nastaví stav na „spracované“, uloží odpoveď a cestu k výslednému súboru. Pri ukladaní výsledkov sa do názvu súboru vkladá aj časová značka, aby bolo možné spätne identifikovať, kedy bol obrázok spracovaný.

- **Zobrazenie výsledkov**

Používateľ si môže následne pozrieť detail fotky, kde sú zobrazené všetky základné informácie o snímke, zoznam metód, ktoré boli na fotku aplikované, a ich aktuálny stav. V prehľadnej tabuľke sú zobrazené aj všetky spracované obrázky vrátane informácií o metóde, stave, dátume vytvorenia a dátume spracovania. Po kliknutí na konkrétny spracovaný obrázok sa zobrazí detail, kde je možné porovnať originálny a spracovaný obrázok, zobraziť podrobné informácie o spracovaní a v prípade potreby si pozrieť aj surové JSON dáta výsledku.

4 Architektúra frontendu

Frontend aplikácia je rozdelená do troch hlavných častí. Šablóny a komponenty definujú základnú HTML štruktúru a znovupoužiteľné časti šablón. Layouty sú implementované pomocou Bootstrap grid systému a dynamický obsah je generovaný pomocou Jinja2 šablón.

Štýlovanie je realizované pomocou Bootstrap 5.3.2 pre základné komponenty a custom CSS pre špecifické komponenty. Pre layout sa používa kombinácia CSS Grid a Flexbox, čo umožňuje vytvárať responzívne a flexibilné rozhranie. JavaScript kód je organizovaný do modulov s použitím ES6+ syntaxe a Fetch API pre HTTP požiadavky.

- **Komponenty**

Frontend aplikácia obsahuje niekoľko hlavných komponentov. Autentifikačný systém zahŕňa prihlasovací a registračný formulár s validáciou v reálnom čase. Dashboard poskytuje prehľad aktivít a základných štatistík. Správa dát je realizovaná prostredníctvom CRUD operácií s možnosťou filtrovania a vyhľadávania.

Zobrazenie obrazov je implementované ako galéria s možnosťami filtrovania a triedenia. Detailné zobrazenie obrazu umožňuje základné operácie ako zoom a pan. Výsledky spracovania sú zobrazované v prehľadnej forme s možnosťou exportu.

- **Responzívny dizajn**

Implementácia responzívneho dizajnu je založená na mobile-first prístupe. Používajú sa štandardné breakpointy (600px, 900px) pre rôzne veľkosti obrazoviek. Typografia je fluidná a obrázky sú responzívne. Implementované je základné spracovanie dotykových udalostí pre mobilné zariadenia.

- **Výkon**

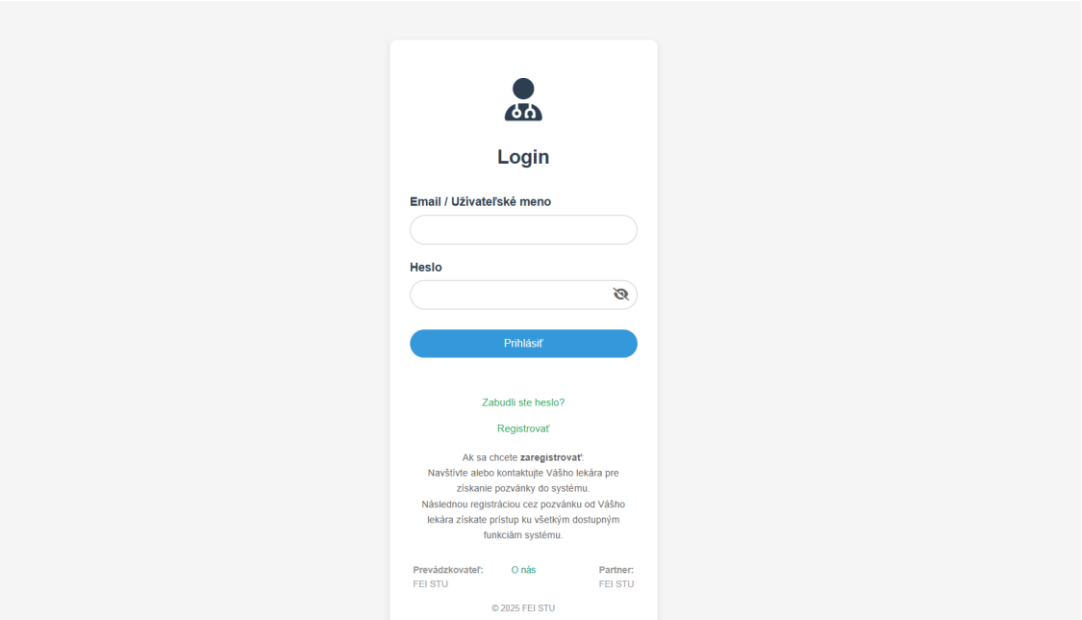
Aktuálne sú implementované základné optimalizácie výkonu. Lazy loading obrázkov v galérii zlepšuje počiatočné načítanie stránky. Statické súbory sú minifikované a používa sa browser caching pre statické assets. Tieto optimalizácie zlepšujú celkový výkon aplikácie a používateľský zážitok.

- **Backend integrácia**

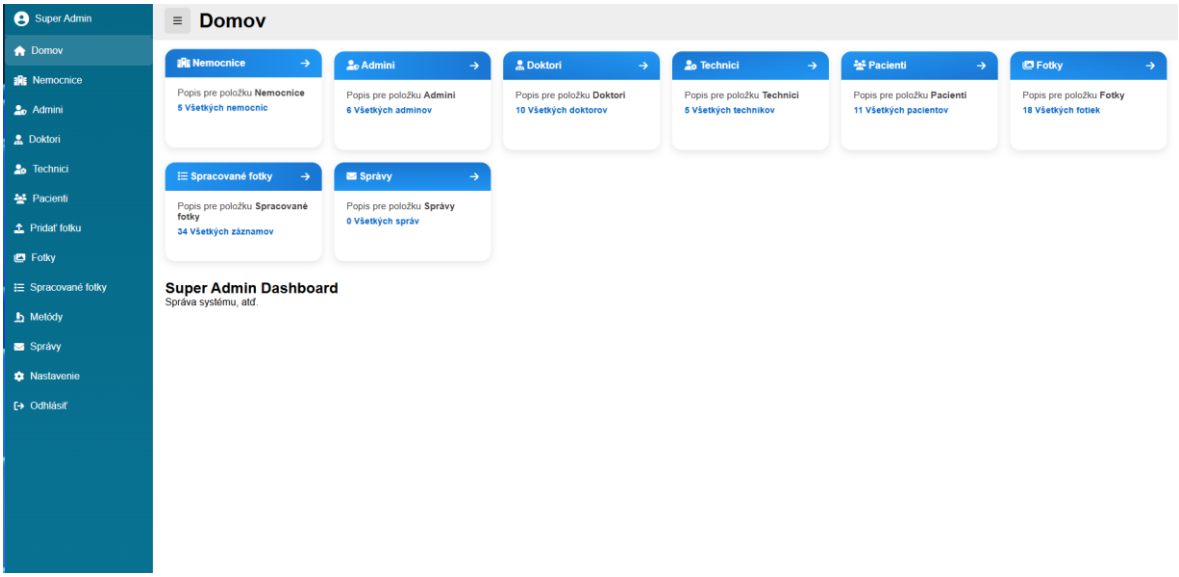
Komunikácia s backendom je realizovaná prostredníctvom REST API endpointov. Dáta sú vymieňané v JSON formáte. Implementovaný je základný error handling a správa stavu načítavania. Offline synchronizácia dát nie je momentálne implementovaná.

5 Screenshoty z aplikácie

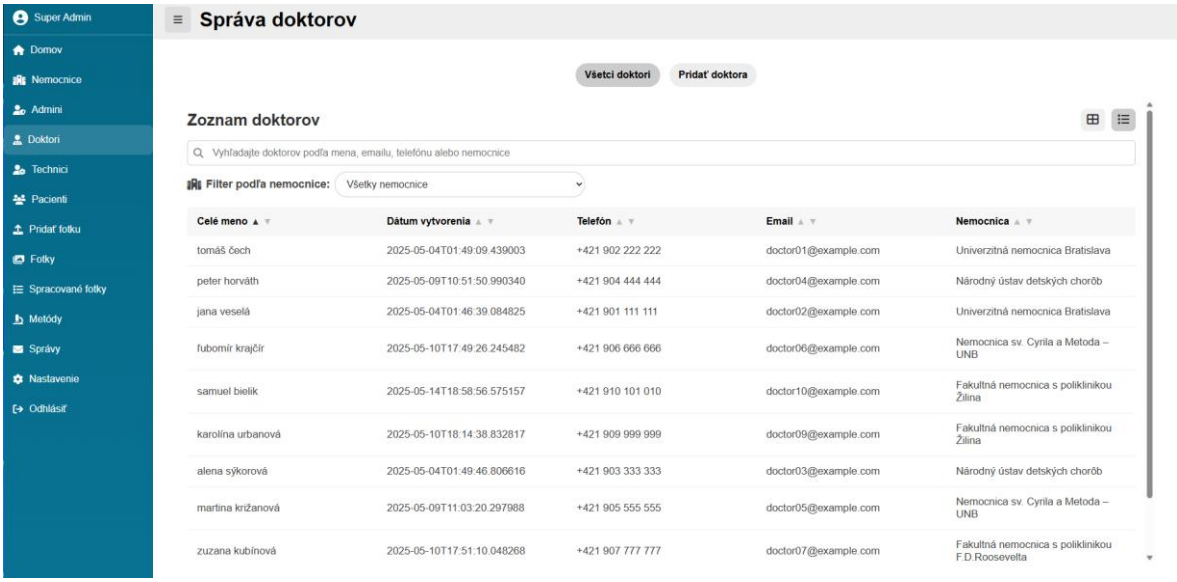
– **Prihlásenie**



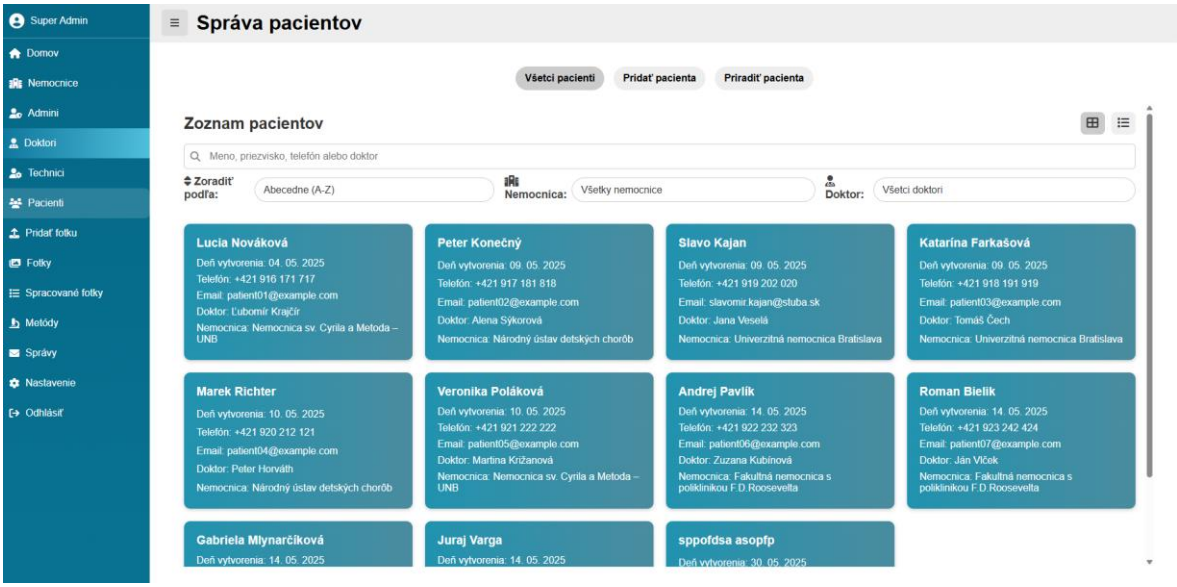
– **Admin dashboard**



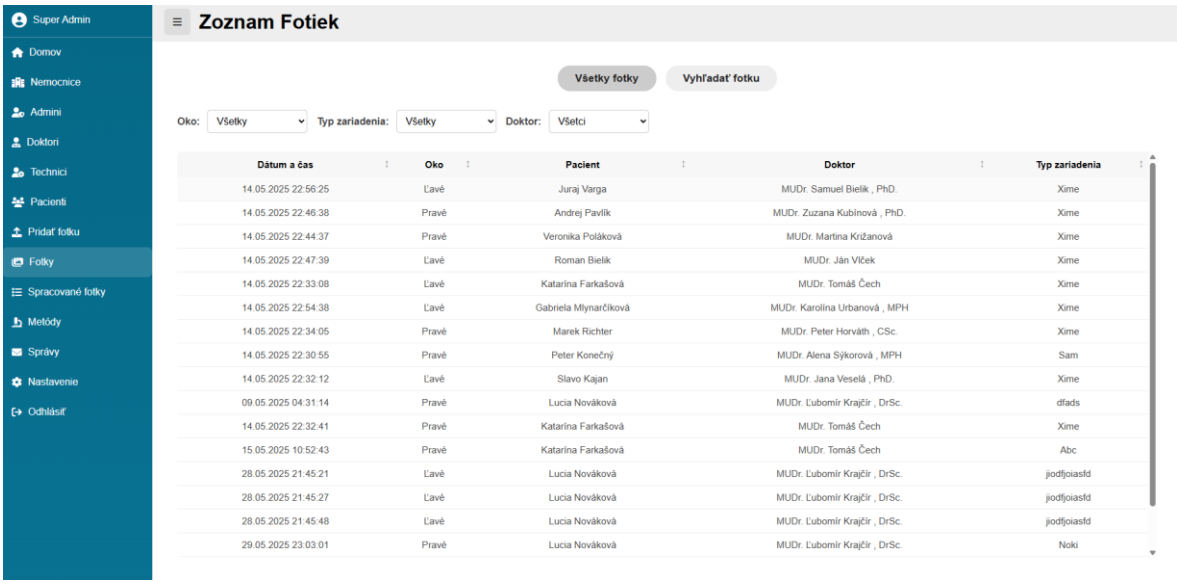
– **Správa doktorov (list)**



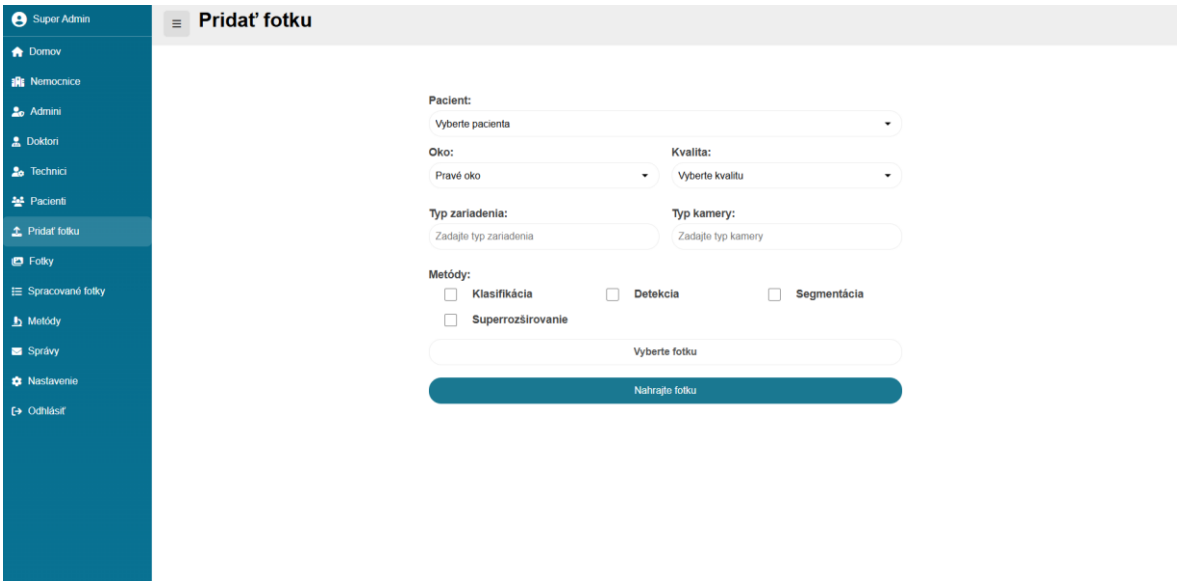
– **Správa pacientov (tabs)**



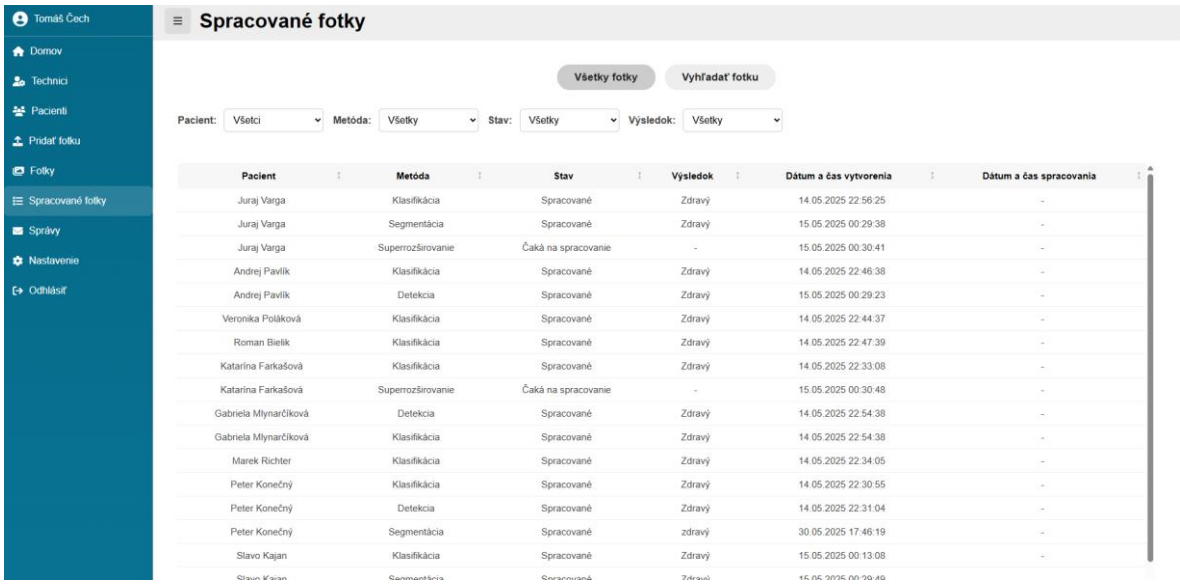
- **Databáza nahraných fotiek**



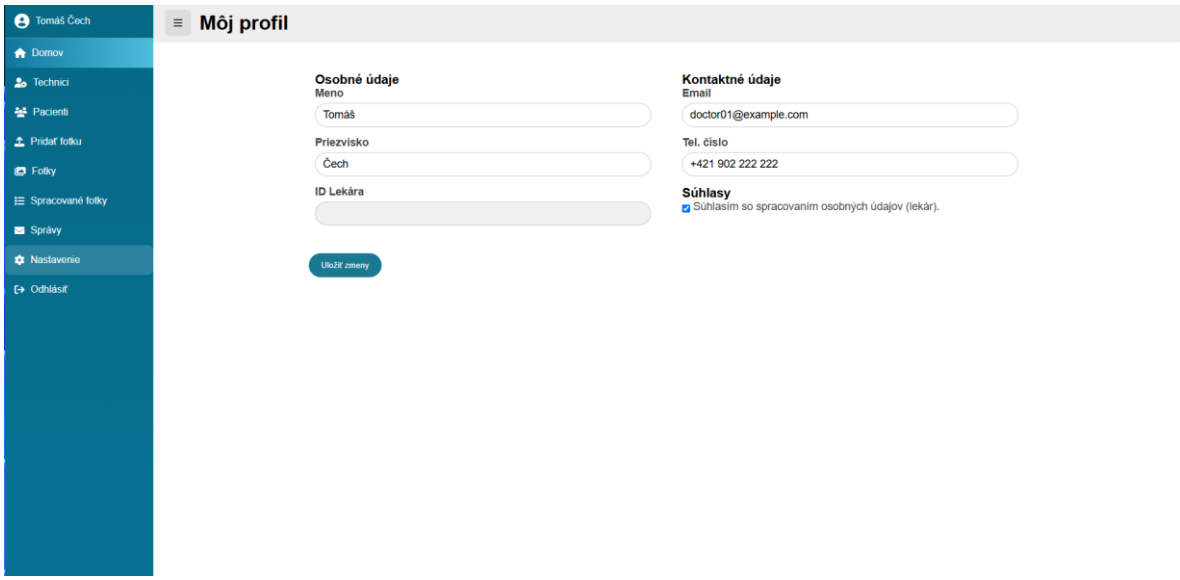
– **Nahrávanie fotiek**



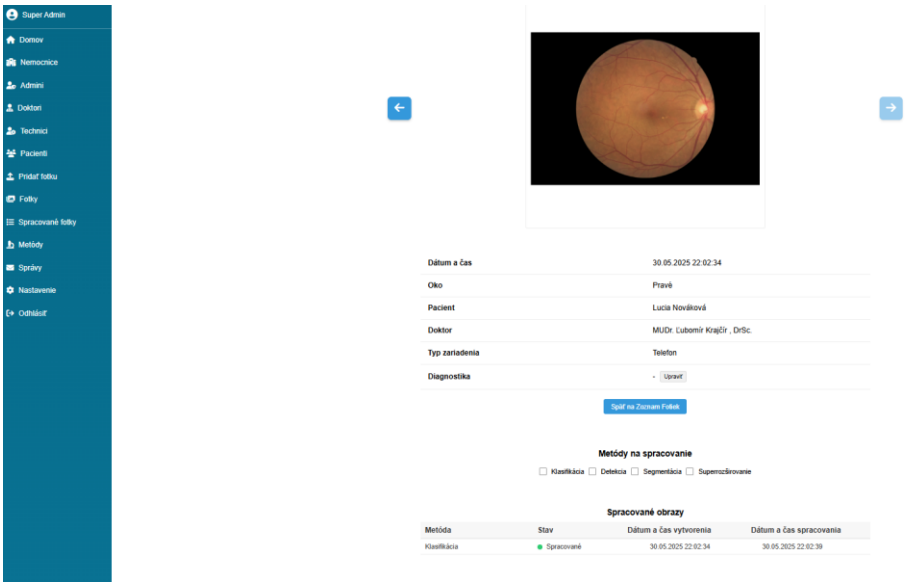
– **Databaza spracovaných fotiek**



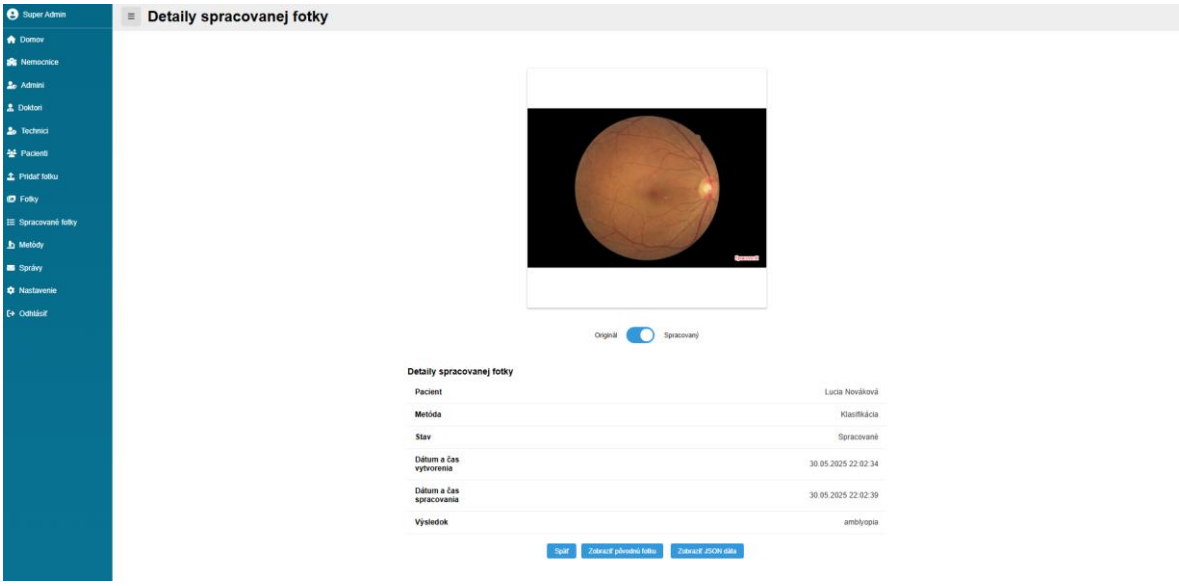
– **Nastavenia uživateľa**



– **Detail fotky**



– **Detail spracovanej fotky**



6 Bezpečnosť

- **Aktuálne implementované opatrenia**

Systém implementuje základné bezpečnostné opatrenia na viacerých úrovniach. Autentifikácia je založená na JWT tokenoch s expiráciou 1 hodina. Heslá sú bezpečne hashované pomocou bcrypt algoritmu. Implementovaný je základný rate limiting pre ochranu pred nadmerným zaťažením.

Autorizácia je realizovaná pomocou role-based access control (RBAC) systému. Každá rola má definované oprávnenia pre prístup k rôznym častiam systému.

Bezpečnosť dát je zabezpečená pomocou TLS 1.3 pre HTTPS komunikáciu. Implementovaná je základná validácia vstupov na API endpointoch a ochrana proti SQL injection. CSRF ochrana je implementovaná pre všetky formuláre.

- **Plánované bezpečnostné vylepšenia**

V budúcnosti sa plánuje implementácia pokročilých bezpečnostných opatrení. Medzi ne patrí refresh token mechanizmus, dvojfaktorová autentifikácia a pokročilý rate limiting. V oblasti autorizácie sa plánuje implementácia granulárnych oprávnení a pokročilého audit logingu.

Pre bezpečnosť dát sa plánuje implementácia AES-256 šifrovania citlivých dát a pokročilá anonymizácia. V oblasti monitoringu sa plánuje implementácia security logging systému a alerting systému. Tiež sa plánuje implementácia nástrojov pre GDPR compliance a pravidelné security review.

7 Inštalácia a konfigurácia

- **Systémové požiadavky**

Pred samotným nasadením treba zabezpecíť, aby hostiteľský server spĺňal minimálnu hardvérovú konfiguráciu: procesor s podporou virtualizácie, aspoň 4 GB RAM a 20 GB voľného miesta na SSD disku pre rýchly I/O. Softvérovo je vyžadovaný Python 3.8 + s nainštalovaným modulom virtualenv a PostgreSQL 12 + s povolenou archiváciou WAL (Write-Ahead Logging) pre zálohovanie a replikáciu.

- **Inštalácia**

1. Klonovanie repozitára
2. Vytvorenie virtuálneho prostredia
3. Inštalácia závislostí z requirements.txt
4. Konfigurácia databázy

- V PostgreSQL vytvorte novú databázu a používateľa s plnými právami.
- V koreňovom adresári zmeňte env_example na .env a nastavte: DATABASE_URL=postgresql://username:password@db-host:5432/retina_db

5. Spustenie migračných skriptov

- **Konfigurácia**

- Nastavenie premenných prostredia v .env súbore
- Konfigurácia databázového pripojenia
- Nastavenie bezpečnostných kľúčov
- Konfigurácia logovania

8 Záver

Projekt webovej aplikácie na spracovanie obrazov očných sietnic priniesol robustné, bezpečné a škálovateľné riešenie, ktoré spĺňa požiadavky klinickej praxe aj výskumných tímov. Vďaka trojvrstvovej architektúre (frontend – Flask backend – PostgreSQL) sme dosiahli jasné oddelenie zodpovedností, jednoduchú udržiavateľnosť a možnosť priebežného rozširovania funkcií. Kritické časti systému, ako je nahrávanie a ukladanie snímok, spracovanie AI-algoritmami, manažment používateľských rolí či auditné logovanie, sú implementované v samostatných moduloch so zreteľným API kontractom.

Z pohľadu výkonu sme aplikáciu optimalizovali pomocou lazy-loadingu obrázkov, minifikácie a cache-ovania statických assetov, čo umožňuje plynulú prácu aj pri veľkom objeme dát. Konfigurácia databázových poolov a indexovanie najfrekventovanejších tabuľkových polí garantuje rýchle odpovede na dotazy a vysokú dostupnosť. Bezpečnostné mechanizmy (TLS 1.3, JWT autentifikácia, RBAC, CSRF, bcrypt) zaručujú ochranu citlivých lekárskych dát v súlade s GDPR.

Modulárny dizajn a využitie overených open-source technológií (Bootstrap, Jinja2, Flask-SQLAlchemy, Alembic, PostgreSQL) poskytujú spoľahlivý základ pre ďalší rozvoj – či už ide o pridanie nových AI-modelov, integráciu s externými informačnými systémami nemocníc alebo rozšírenie archivalnych a reportingových funkcií. V horizonte budúcna plánujeme zaviesť pokročilé bezpečnostné vrstvy (2FA, refresh tokeny), detailnejšiu telemetriu výkonu a automatizované testovanie end-to-end.

Výsledkom je ucelený nástroj, ktorý znižuje manuálnu záťaž lekárov a technikov, minimalizuje chyby pri spracovaní snímok a prináša konzistentné, auditovateľné dáta pre podporu diagnózy. Aplikácia tak významne prispieva k efektívnejšej a bezpečnejšej práci s medicínskymi obrazmi, pričom jej architektúra a implementácia sú dostatočne flexibilné na to, aby sa v budúcnosti dokázala prispôsobiť meniacim sa technologickým i regulačným požiadavkám.